

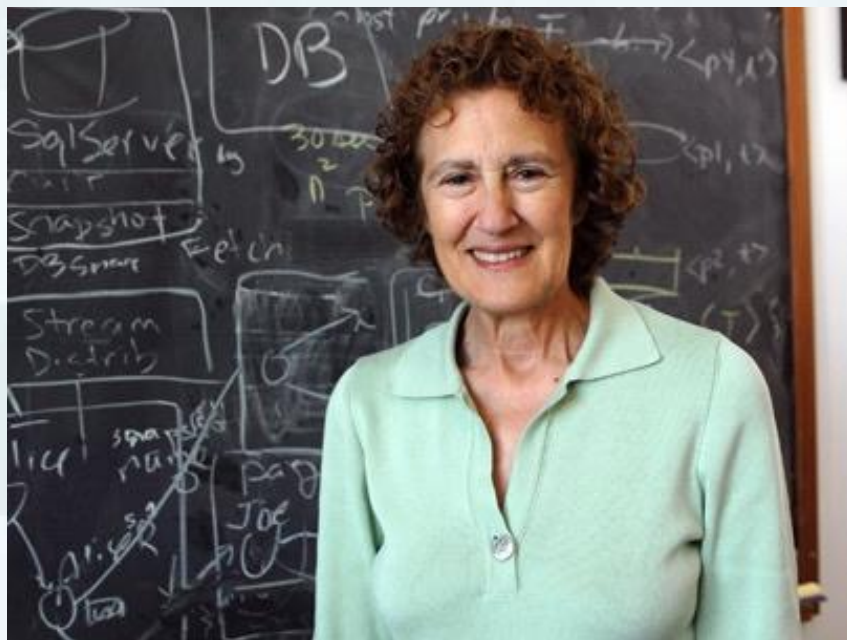


里氏替换原则

Liskov Substitution Principle, LSP

名字的由来

- 名字的由来（Liskov Substitution Principle, LSP）
- 里氏代换原则以 **Barbara Liskov**（芭芭拉·利斯科夫）教授的姓氏命名。最早是在1988年，由麻省理工学院的女教授（芭芭拉·利斯科夫）提出来的。
- 芭芭拉·利斯科夫：美国计算机科学家，2008年图灵奖得主，2004年约翰·冯诺依曼奖得主，美国工程院院士，美国艺术与科学院院士，美国计算机协会会士，麻省理工学院电子电气与计算机科学系教授，美国第一位计算机科学女博士。





目录

4

使用该原则
注意事项

3

里氏替换
原则包含的
含义

2

里氏替换
原则的定义

1

继承的弊端



一、继承的弊端

继承作为面向对象三大特性之一，在给程序设计带来巨大便利的同时，也带来了弊端。例如：

- 1、继承是侵入性的。只要继承，就必须拥有父类的所有属性和方法；
- 2、降低代码的灵活性。子类必须拥有父类的属性和方法，让子类自由的世界中多了些约束；
- 3、增强了耦合性。当父类的常量、变量和方法被修改时，必需要考虑子类的修改，而且在缺乏规范的环境下，这种修改可能带来非常糟糕的结果——大片的代码需要重构。



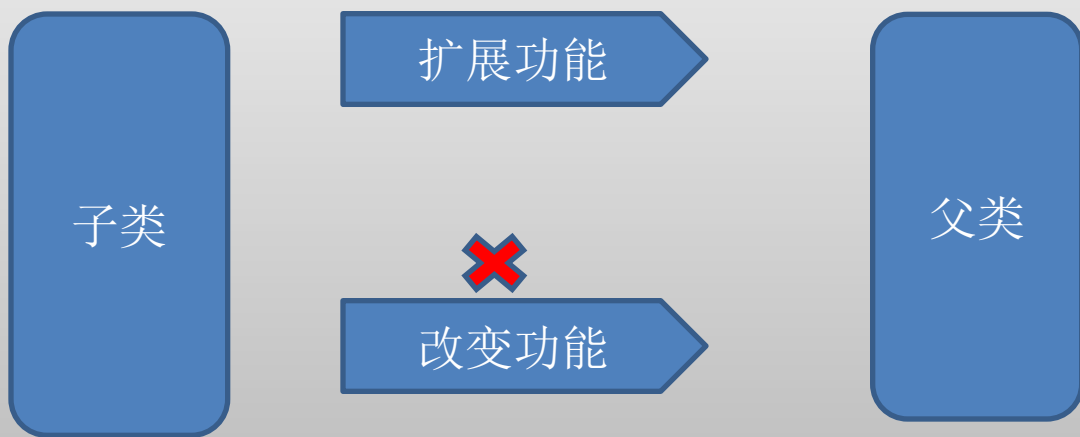
一、继承的弊端

问题：怎样才能让继承机制的“利”大于“弊”？

 引入里氏替换原则

二、里氏替换原则的定义

1、里氏替换原则通俗的来讲就是：子类可以扩展父类的功能，但不能改变父类原有的功能。





二、里氏替换原则的定义

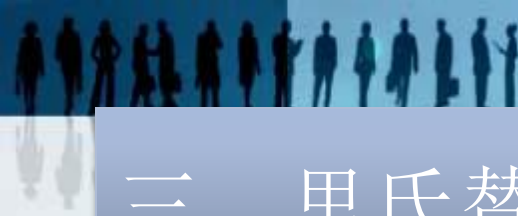
2、里氏代换原则告诉我们，在软件中将一个基类对象替换成它的子类对象，程序将不会产生任何错误和异常，反过来则不成立，如果一个软件实体使用的是一个子类对象的话，那么它不一定能够使用基类对象。

JAVA中：基类称为父类，导出类称为子类



二、里氏替换原则的定义

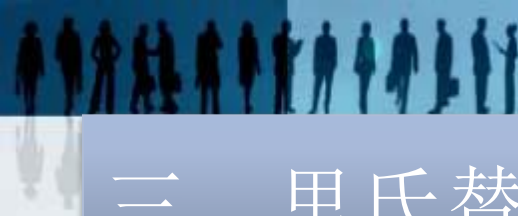
3、里氏代换原则是实现开闭原则的重要方式之一，由于使用基类对象的地方都可以使用子类对象，因此在程序中尽量使用基类类型来对对象进行定义，而在运行时再确定其子类类型，用子类对象来替换父类对象。



三、里氏替换原则包含的含义

1、子类可以实现父类的抽象方法，但是不能覆盖父类的非抽象方法

在我们做系统设计时，经常会设计接口或抽象类，然后由子类来实现抽象方法，这里使用的其实就是里氏替换原则。子类可以实现父类的抽象方法很好理解，事实上，子类也必须完全实现父类的抽象方法，哪怕写一个空方法，否则会编译报错。



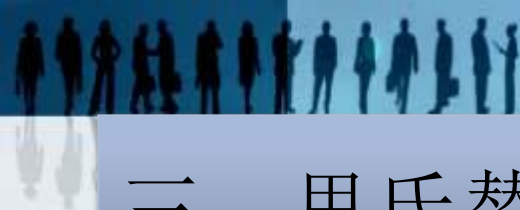
三、里氏替换原则包含的含义

里氏替换原则的**关键点**在于**不能覆盖父类的非抽象方法**

。

父类中凡是已经实现好的方法，实际上是在设定一系列的规范和契约，虽然它不强制要求所有的子类必须遵从这些规范，但是如果子类对这些非抽象方法任意修改，就会对整个继承体系造成破坏。而里氏替换原则就是表达了这一层含义

。



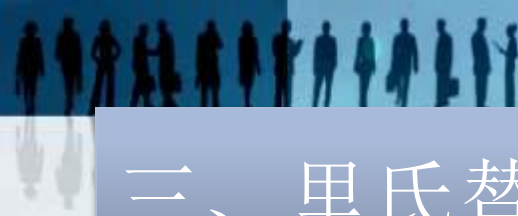
三、里氏替换原则包含的含义

在面向对象的设计思想中，继承这一特性为系统的设计带来了极大的便利性，但是由之而来的也潜在着一些风险。所以，子类**C1**继承父类**C**时，可以添加新方法完成新增功能，尽量不要重写父类**C**的方法。否则可能带来难以预料的风险。下面举例说明：



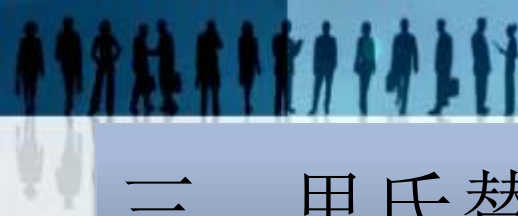
```
public class C {  
    public int func(int a, int b) {  
        return a+b;  
    }  
}  
  
public class C1 extends C {  
    @Override  
    public int func(int a, int b) {  
        return a-b;  
    }  
}  
  
public class Client {  
    public static void main(String[] args) {  
        C c = new C1();  
        System.out.println("2+1=" + c.func(2, 1));  
    }  
}
```

运行结果: 2+1=1



三、里氏替换原则包含的含义

上面的运行结果明显是错误的。类**C1**继承**C**，后来需要增加新功能，类**C1**并没有新写一个方法，而是直接重写了父类**C**的**func**方法，违背里氏替换原则，引用父类的地方并不能透明的使用子类的对象，导致运行结果出错。



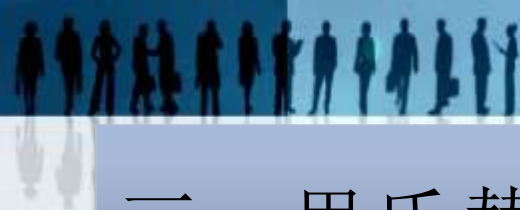
三、里氏替换原则包含的含义

2、子类中可以增加自己特有的方法

在继承父类属性和方法的同时，每个子类也都可以有自己的个性，在父类的基础上扩展自己的功能。前面其实已经提到，当功能扩展时，子类尽量不要重写父类的方法，而是另写一个方法，所以对上面的代码加以更改，使其符合里氏替换原则。如下例所示：

```
public class C {  
    public int func(int a, int b) {  
        return a+b;  
    }  
}  
  
public class C1 extends C {  
    public int func2(int a, int b) {  
        return a-b;  
    }  
}  
  
public class Client {  
    public static void main(String[] args) {  
        C1 c = new C1();  
        System.out.println("2-1=" + c.func2(2, 1));  
    }  
}
```

运行结果：2-1=1



三、里氏替换原则包含的含义

3、当子类覆盖或实现父类的方法时，方法的前置条件（即方法的形参）要比父类方法的输入参数更宽松



```
import java.util.HashMap;
public class Father {
    public void func(HashMap m) {
        System.out.println("执行父类...");
    }
}
```

HashMap
implements Map

```
import java.util.Map;
public class Son extends Father{
    public void func(Map m) { //方法的形参比父类的更宽松
        System.out.println("执行子类...");
    }
}
```

Map是接口

```
import java.util.HashMap;
public class Client{
    public static void main(String[] args) {
        Father f = new Son(); //引用基类的地方能透明地使用其子类的对象。
        HashMap h = new HashMap();
        f.func(h);
    }
}
```

HashMap 是Map
的实现类



三、里氏替换原则包含的含义

运行结果：执行父类...

注意Son类的func方法前面是不能加@Override注解的，因为否则会编译提示报错，因为这并不是重写（Override），而是重载（Overload），因为方法的输入参数不同。



三、里氏替换原则包含的含义

4、当子类的方法实现父类的抽象方法时，方法的后置条件（即方法的返回值）要比父类更严格



```
import java.util.Map;
public abstract class Father {
    public abstract Map func();
}

import java.util.HashMap;
public class Son extends Father{

    @Override
    public HashMap func() { //方法的返回值比父类的更严格
        HashMap h = new HashMap();
        h.put("h", "执行子类...");
        return h;
    }
}

public class Client{
    public static void main(String[] args) {
        Father f = new Son(); //引用基类的地方能透明地使用其子类的对象。
        System.out.println(f.func());
    }
}
```

HashMap
implements Map

Map是接口

HashMap 是Map
的实现类

执行结果: {h=执行子类...}



四、在使用里氏代换原则时需要注意如下几个问题

1、子类的所有方法必须在父类中声明，或子类必须实现父类中声明的所有方法。根据里氏代换原则，为了保证系统的扩展性，在程序中通常使用父类来进行定义，如果一个方法只存在于子类中，在父类中不提供相应的声明，则无法在以父类定义的对象中使用该方法。



四、在使用里氏代换原则时需要注意如下几个问题

2、我们在运用里氏代换原则时，**尽量把父类设计为抽象类或者接口，让子类继承父类或实现父接口，并实现在父类中声明的方法**，运行时，子类实例替换父类实例，我们可以很方便地扩展系统的功能，同时无须修改原有子类的代码，增加新的功能可以通过增加一个新的子类来实现。里氏代换原则是开闭原则的具体实现手段之一。

3、在系统设计时，遵循里氏替换原则，**尽量避免子类重写父类的方法**，可以有效降低代码出错的可能性。

在Sunny软件公司开发的CRM系统中，客户(Customer)可以分为VIP客户(VIPCustomer)和普通客户(CommonCustomer)两类，系统需要提供一个发送Email的功能，原始设计方案如图1所示：

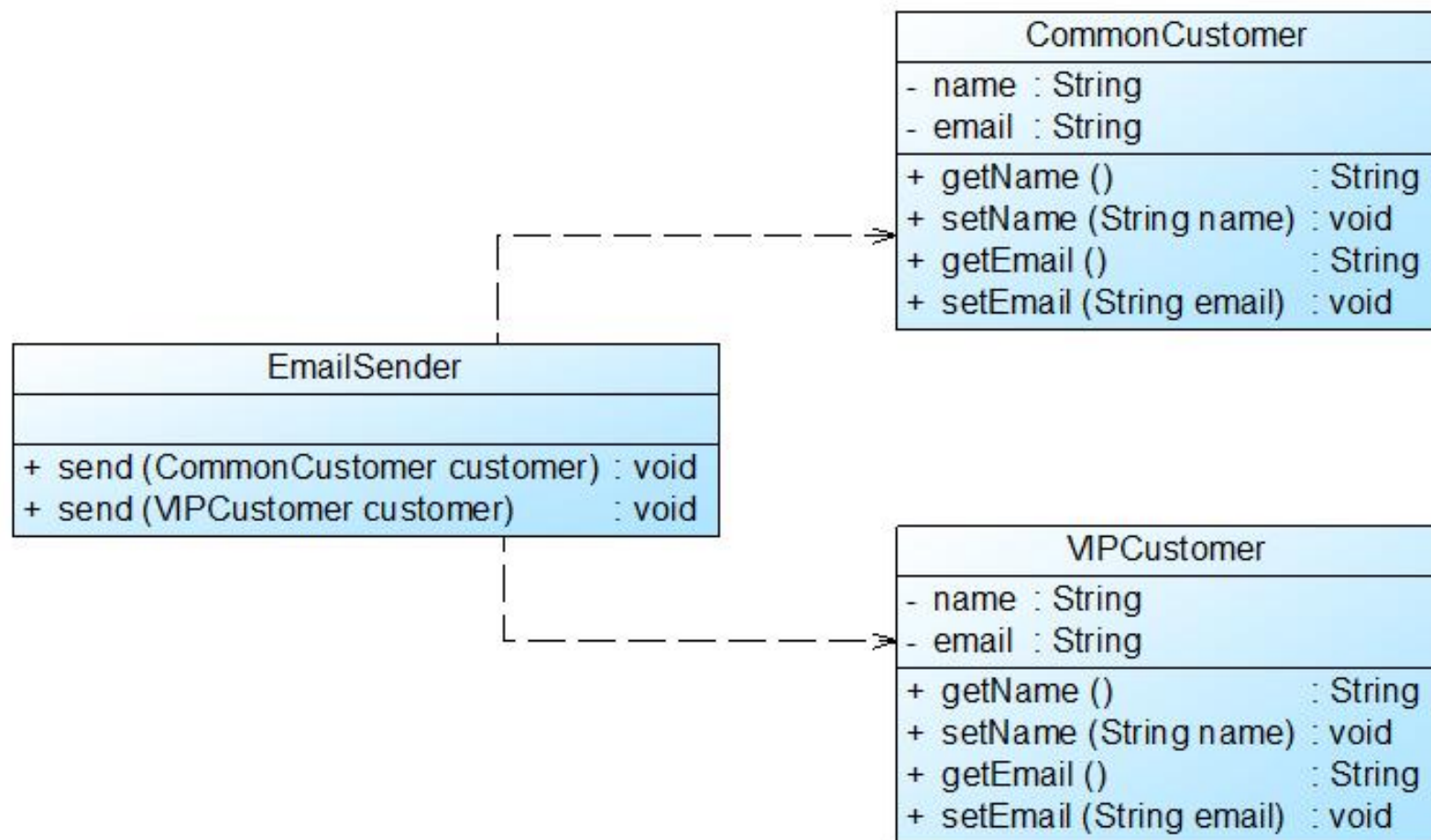


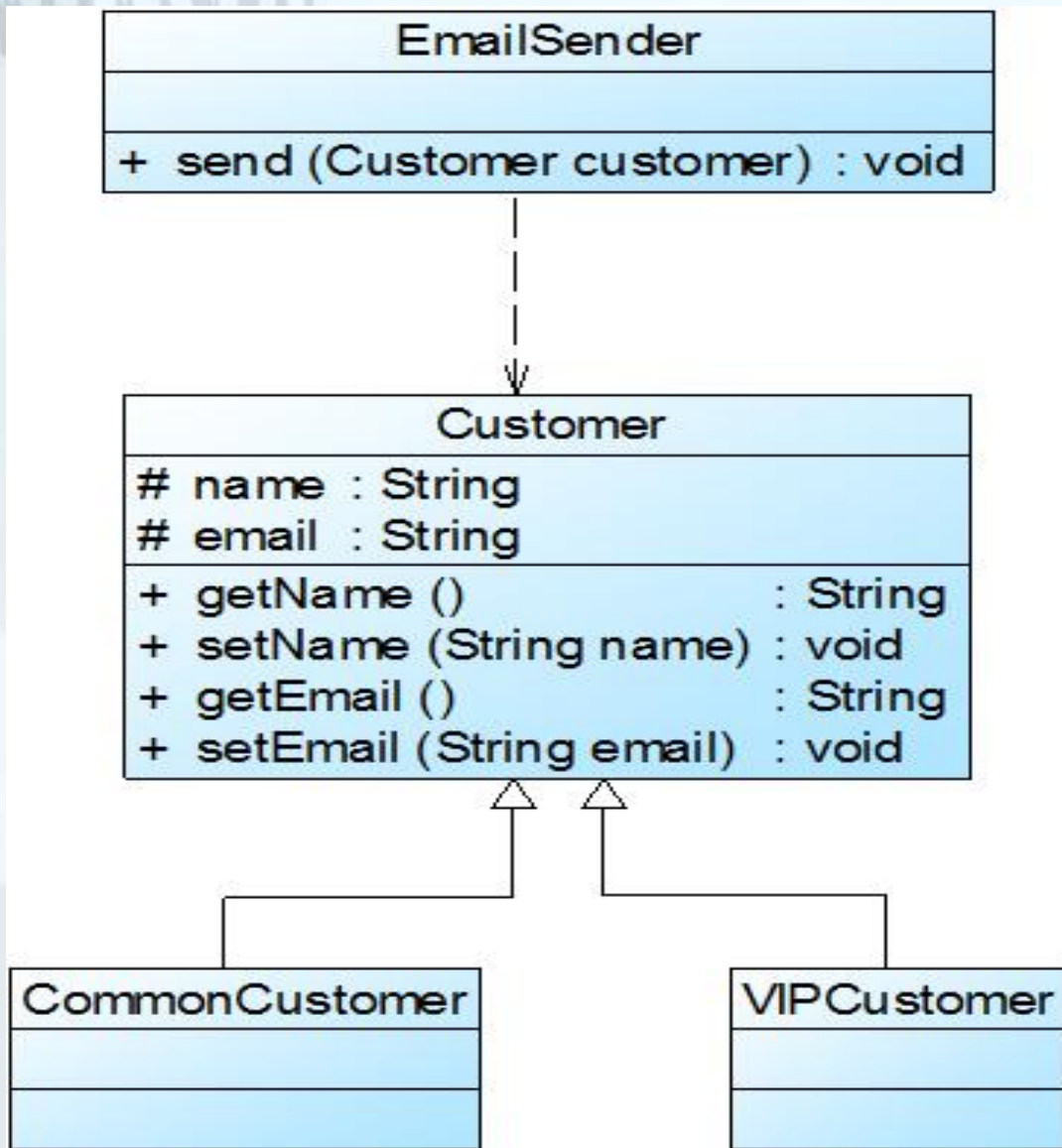
图1原始结构图



在对系统进行进一步分析后发现，无论是普通客户还是VIP客户，发送邮件的过程都是相同的，也就是说两个 `send()` 方法中的代码重复，而且在本系统中还将增加新类型的客户。为了让系统具有更好的扩展性，同时减少代码重复，使用里氏代换原则对其进行重构。



在本实例中，可以考虑增加一个新的抽象客户类Customer，而将CommonCustomer和VIPCustomer类作为其子类，邮件发送类EmailSender类针对抽象客户类Customer编程，根据里氏代换原则，**能够接受基类对象的地方必然能够接受子类对象**，因此将EmailSender中的send()方法的参数类型改为Customer，如果需要增加新类型的客户，只需将其作为Customer类的子类即可。重构后的结构如图2所示：



知识点:

1、尽量把父类设计为抽象类或者接口

2、能够接受基类对象的地方必然能够接受子类对象



谢谢观看！

